CECyT 9 · Programación Web

# Reporte de práctica de Postman

API REST de talleres, asistencias y usuarios

**Alumno**

**Vertiz Angeles Jose Elias Caleb**

Boleta: 2025090784

Fecha: 27 de octubre de 2026

**Entregables**

coleccion-2025090784.json

reporte-2025090784.pdf

Backend: http://localhost:4000

BD: MySQL · mierda_batiz

## Resultado general

**24 de 24 peticiones respondieron el código esperado.**

Se incluyen las 4 capturas solicitadas: crear taller, listar talleres, crear usuario y boleta corta.

Datos de prueba: taller id 8 “Taller de prueba Postman” · usuario id 19 “Alumno de prueba Postman” · boleta 2025090784.

## Tabla de resultados

| # | Petición | Esperado | Obtuve | Observaciones |
|---|---|---|---|---|
| 1 | **GET /saludo** | 200 | 200 | La guía propone GET /; el backend real expone la ruta de vida en /saludo. Se probó esa ruta. |
| 2 | **GET /api/talleres** | 200 | 200 | Devuelve el arreglo completo de talleres. |
| 3 | **GET /api/talleres/:id** | 200 | 200 | Probado con el taller 8, creado desde Postman. |
| 4 | **GET /api/talleres/:id inexistente** | 404 | 404 | Responde Usuario/Taller no encontrado; no rompe el servidor. |
| 5 | **POST /api/talleres** | 201 | 201 | Crea el taller 8. Evidencia en la captura 01. |
| 6 | **POST /api/talleres sin nombre** | 400 | 400 | Valida nombre y fecha obligatorios. |
| 7 | **POST /api/asistencias** | 201 | 201 | Registra la asistencia al taller 8 con la boleta 2025090784. |
| 8 | **GET /api/talleres/:id/asistencias** | 200 | 200 | Lista las asistencias del taller 8. |
| 9 | **POST /api/asistencias incompleta** | 400 | 400 | Valida taller_id y boleta. |
| 10 | **POST /api/asistencias taller inexistente** | 500 | 500 | La FK de MySQL lanza error y el controlador lo devuelve como 500. Ver respuesta 3. |
| 11 | **GET /api/usuarios** | 200 | 200 | Devuelve la lista de usuarios sin contraseñas. |
| 12 | **POST /api/usuarios** | 201 | 201 | Crea el usuario 19 con correo y boleta nuevos. Evidencia en la captura 03. |

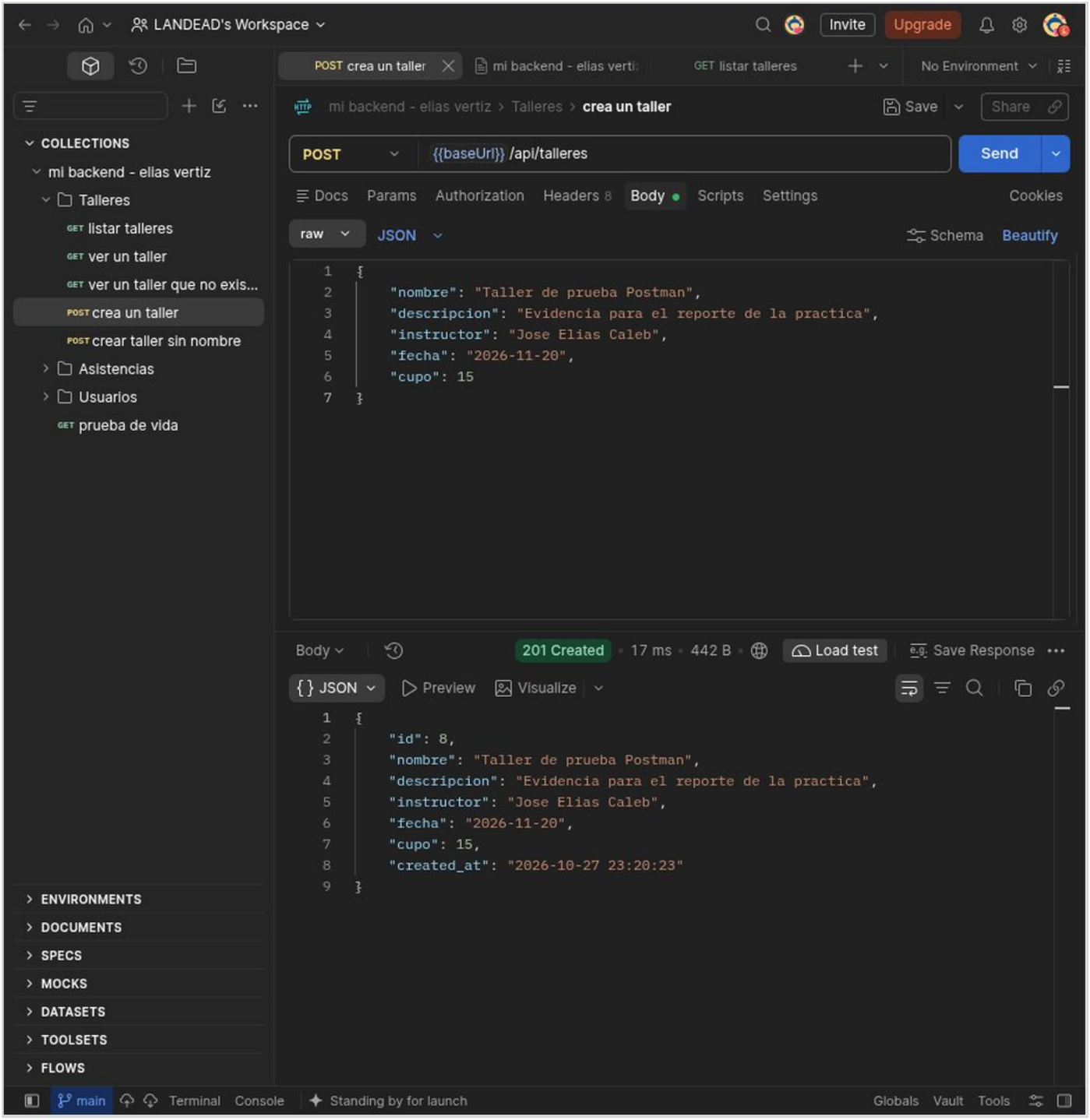
## Tabla de resultados (continuación)

| # | Petición | Esperado | Obtuve | Observaciones |
|---|---|---|---|---|
| 13 | **GET /api/usuarios/:id** | 200 | 200 | Probado con el usuario 19. |
| 14 | **GET /api/usuarios/:id inexistente** | 404 | 404 | Responde Usuario no encontrado. |
| 15 | **POST /api/usuarios repetido** | 409 | 409 | La boleta/correo ya existen; se detecta ER_DUP_ENTRY. |
| 16 | **POST /api/usuarios correo inválido** | 400 | 400 | validarCorreo rechaza el formato. |
| 17 | **POST /api/usuarios boleta corta** | 400 | 400 | validarBoleta exige 10 dígitos. Evidencia en la captura 04. |
| 18 | **POST /api/usuarios contraseña corta** | 400 | 400 | validarContrasena exige al menos 8 caracteres. |
| 19 | **POST /api/usuarios sin datos** | 400 | 400 | Valida los campos obligatorios. |
| 20 | **PUT /api/usuarios/:id** | 200 | 200 | Actualiza solo el rol del usuario 19. |
| 21 | **PUT /api/usuarios/:id sin datos** | 400 | 400 | Exige al menos nombre, correo o rol. |
| 22 | **PUT /api/usuarios/:id correo repetido** | 409 | 409 | Intenta usar admin@cecyt9.local; se detecta duplicado. |
| 23 | **PUT /api/usuarios/:id inexistente** | 404 | 404 | Responde Usuario no encontrado. |
| 24 | **DELETE /api/usuarios/:id** | 204 | 204 | Se usó el id 19. La petición guardada apunta a /12; conviene cambiarla a {{usuariold}}. |

## Observaciones y recomendaciones

- La guía propone GET / como prueba de vida, pero el backend implementa GET /saludo. Se probó la ruta real y se anotó en la tabla; GET / no existe en este servidor.
- Para crear el taller y el usuario se usaron datos de prueba nuevos. Las capturas 01 y 03 muestran el body enviado y la respuesta 201; así se evita chocar con registros que ya existían.
- La petición guardada “eliminar un usuario” apunta a /api/usuarios/12; para la prueba se usó el id 19 del usuario creado. Conviene cambiarla a {{usuariold}} para que elimine el usuario actual y no un id fijo.
- Las peticiones “crear usuario” y “crear usuario repetido” conservan el correo y la boleta originales. Antes de una nueva corrida hay que actualizarlos a datos libres; en esta ejecución se usó postman.2025090784@alumno.cecyt9.local y 2025090784.
- El 500 de la petición 10 no es el comportamiento ideal: el controlador deja que la llave foránea de MySQL lance el error. Se explica cómo corregirlo en la respuesta 3.
- El usuario 19 se eliminó al final de la prueba (204); la asistencia creada para el taller 8 se borró en cascada. El taller 8 se conserva como evidencia del listado.

# Captura 1. Crear taller — 201 Created

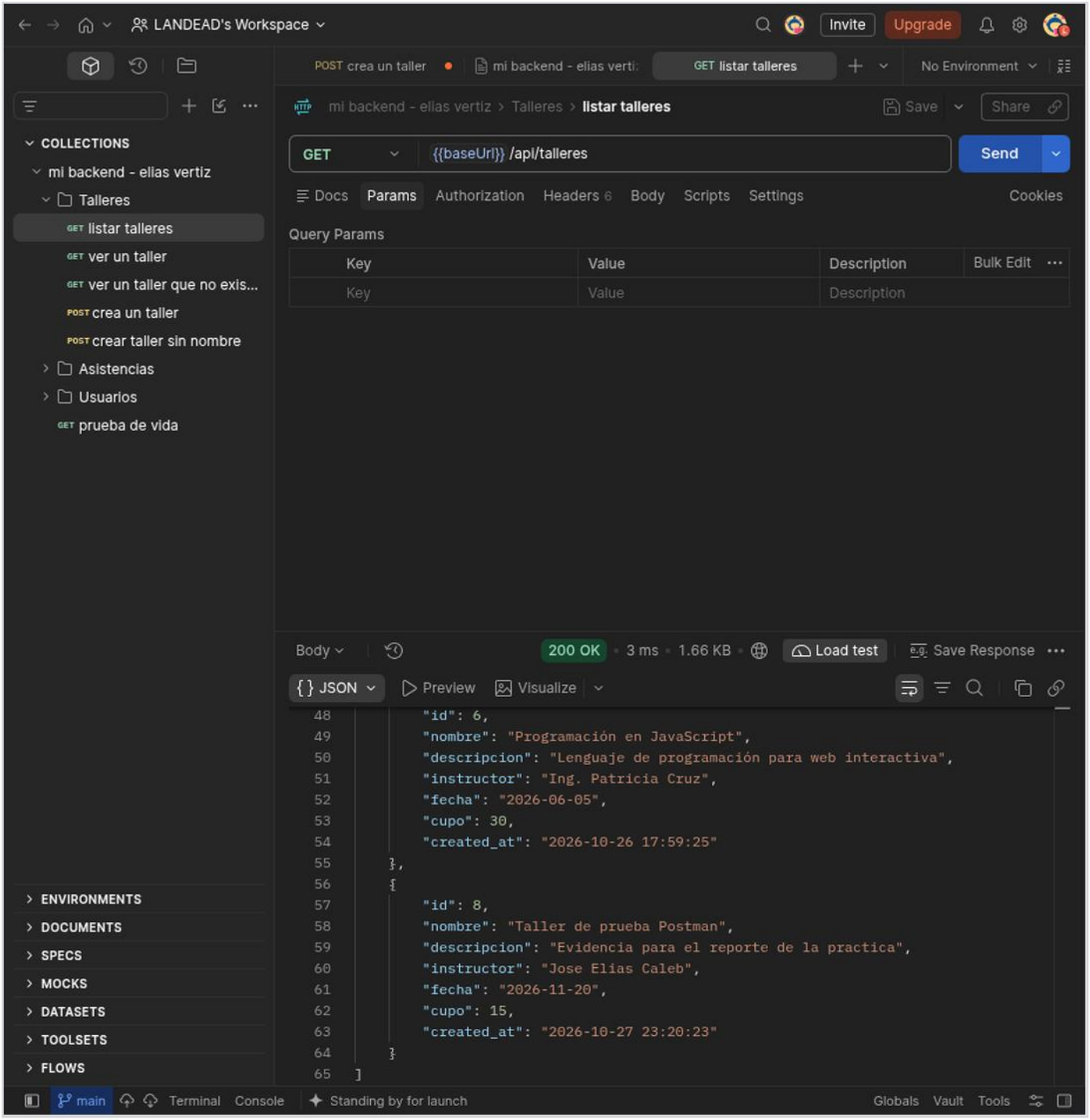



**POST http://localhost:4000/api/talleres**

## ¿Qué se observa?

- Se ve el body JSON con los datos del taller nuevo: nombre, descripción, instructor, fecha y cupo.
- El status es 201 Created y la respuesta devuelve el id 8, generado por la base de datos.
- El taller creado se usó después para las pruebas de listado y de asistencias.

# Captura 2. Listar talleres — aparece el taller nuevo

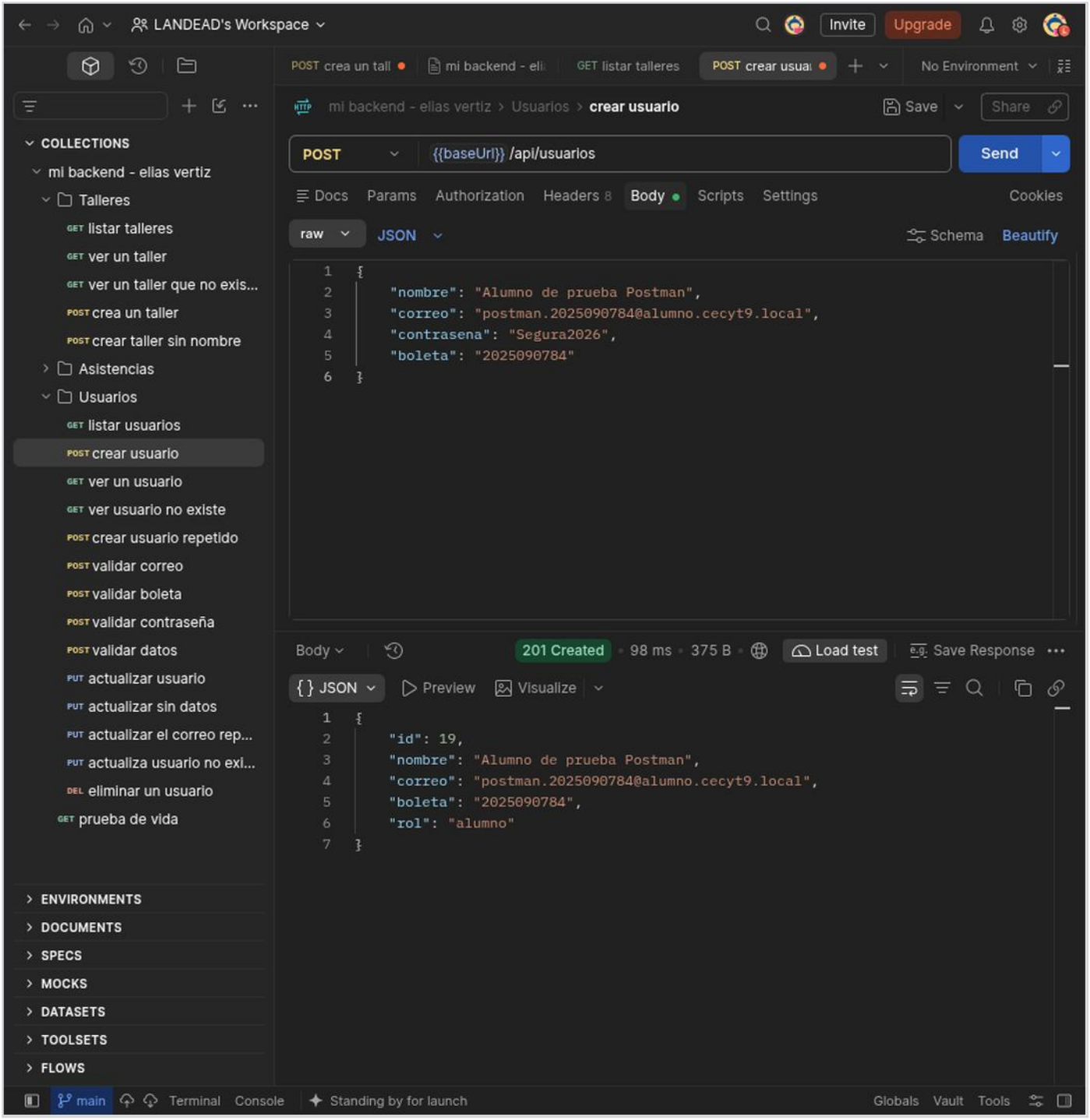



**GET http://localhost:4000/api/talleres**

## ¿Qué se observa?

- El status es 200 OK y el listado incluye el taller recién creado al final: id 8, “Taller de prueba Postman”.
- La respuesta trae los datos guardados en MySQL, incluida la fecha created_at.
- Esto demuestra que el POST anterior sí persistió información y no fue una respuesta simulada.

# Captura 3. Crear usuario — 201 sin contraseña

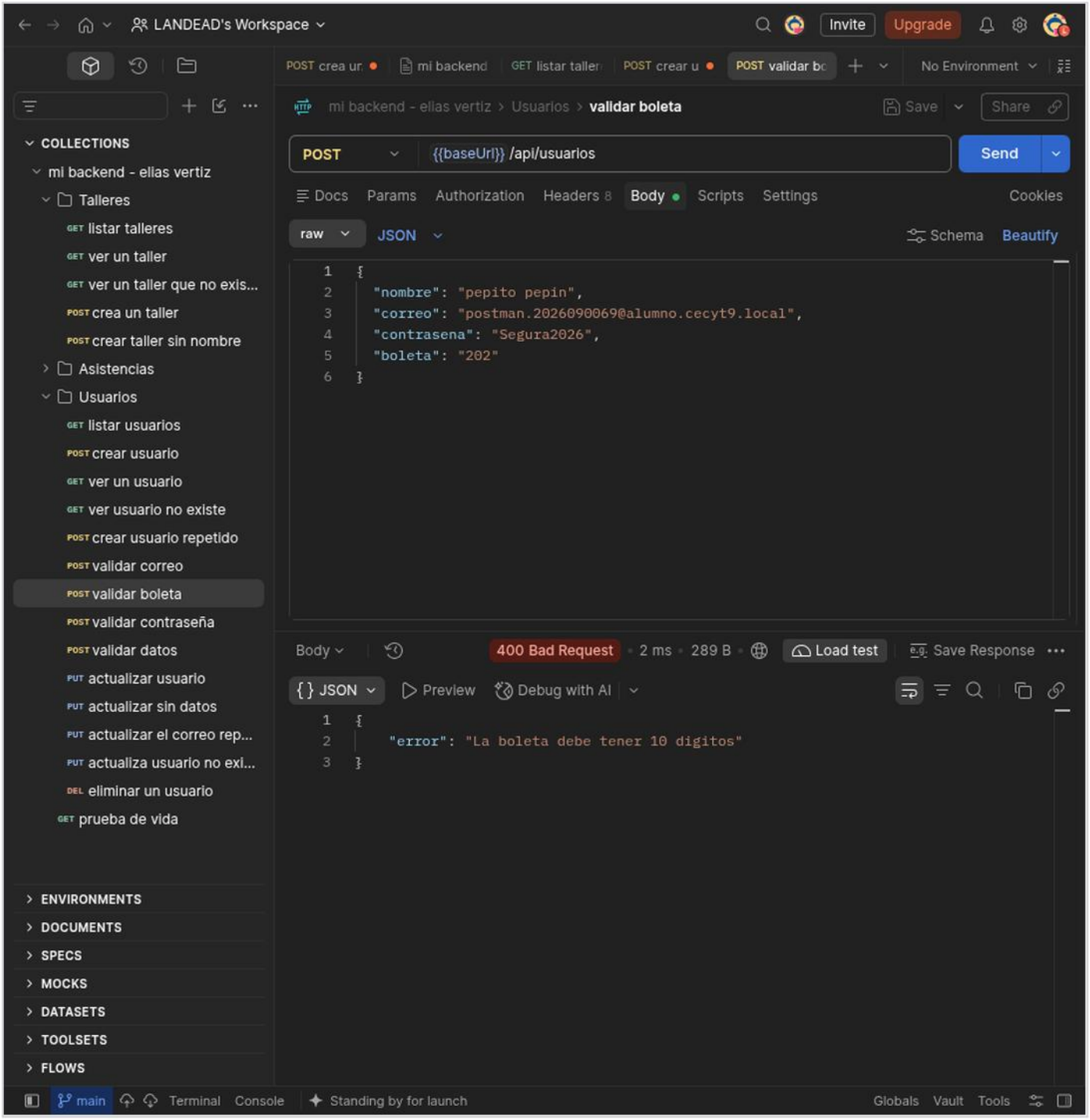



**POST http://localhost:4000/api/usuarios**

## ¿Qué se observa?

- El body envía nombre, correo, contraseña y boleta; el status es 201 Created.
- La respuesta devuelve id 19, nombre, correo, boleta y rol, pero no devuelve la contraseña ni su hash.
- La omisión de la contraseña es correcta: el backend no debe exponer datos sensibles en la respuesta.

# Captura 4. Boleta corta — 400 Bad Request




**POST http://localhost:4000/api/usuarios**

## ¿Qué se observa?

- El body envía la boleta “202”, que no cumple la regla de 10 dígitos.
- El backend responde 400 Bad Request con el mensaje “La boleta debe tener 10 digitos”.
- La validación ocurre en el servidor, aunque la petición no venga del formulario del front.

# Respuestas a las preguntas del reporte

A continuación se responden las cuatro preguntas solicitadas en la guía, usando como referencia las peticiones y respuestas obtenidas en Postman.

## 1. Diferencia entre 200, 201 y 204

Los tres códigos pertenecen a la familia 2xx y significan que la petición fue aceptada, pero se diferencian por el resultado y por el cuerpo de la respuesta:

- 200 OK: la operación se completó y normalmente hay un cuerpo con datos. Se obtuvo en GET /saludo, GET /api/talleres, GET /api/talleres/8, GET /api/talleres/8/asistencias, GET /api/usuarios, GET /api/usuarios/19 y PUT /api/usuarios/19.
- 201 Created: se creó un recurso nuevo y la respuesta incluye el recurso con su id. Se obtuvo en POST /api/talleres (taller 8), POST /api/usuarios (usuario 19) y POST /api/asistencias.
- 204 No Content: la operación se completó, pero no se devuelve cuerpo. Se obtuvo en DELETE /api/usuarios/19.

También aparecieron 400 (datos inválidos), 404 (recurso inexistente), 409 (duplicado) y 500 (error del servidor). No son 2xx porque la petición no se completó como se esperaba.

## 2. ¿Por qué validar la boleta en el backend si el front ya lo hace?

La validación del front es una ayuda para el usuario, no una garantía. Cualquier persona puede saltarse la interfaz y llamar directamente a la API con Postman, curl o desde la consola del navegador; también puede modificar el JavaScript de la página. Por eso el backend debe repetir las validaciones: es la última barrera y la fuente de verdad de los datos.

En la práctica se comprobó con POST /api/usuarios usando la boleta “202”: la petición no pasó por el formulario del front y aun así el backend respondió 400 con el mensaje “La boleta debe tener 10 digitos”. Validar en el servidor también protege la base de datos de datos inconsistentes y de ataques; es defensa en profundidad.

Además, la API puede ser consumida por otros clientes (una app móvil, otro servicio o un script). Si solo se valida en el front, esos clientes podrían guardar datos inválidos. Por eso la regla debe vivir en el backend y, si es posible, también reflejarse en la base de datos.

# Respuestas 3 y 4 · Conclusión

## 3. El 500 al registrar asistencia en un taller inexistente

El 500 es responsabilidad del servidor. El cliente envió un taller_id que no existe, pero un dato inválido no debe provocar un error interno: el servidor debió detectarlo y responder con un 4xx. Lo más correcto es 404 Not Found, porque el recurso taller no existe; si se considera que el cuerpo es inválido, 400 Bad Request también sería aceptable.

Para corregirlo, antes del INSERT se debe consultar el taller y cortar el flujo si no existe:

```
const taller = await talleresModel.obtenerPorId(taller_id);
if (!taller) return res.status(404).json({ error: 'Taller no encontrado' });
```

También conviene capturar el error de llave foránea y mapearlo a 404 o 400 en lugar de dejar que llegue al catch general como 500. Con eso, la petición 10 respondería 404 y el cliente sabría que el taller no existe. Si se acepta usuario_id, debe validarse igual.

## 4. ¿Cómo demostró Postman que los datos son reales?

Postman mostró que los datos son reales porque las operaciones cambiaron el estado de la base de datos y esos cambios se vieron en peticiones posteriores:

- POST /api/talleres respondió 201 y devolvió un id generado (8). Luego GET /api/talleres incluyó ese taller con sus datos y su fecha de creación; la captura 02 lo muestra.
- POST /api/usuarios respondió 201 y devolvió el id 19; GET /api/usuarios/19 devolvió el mismo usuario. La respuesta no incluía la contraseña, solo nombre, correo, boleta y rol.
- DELETE /api/usuarios/19 respondió 204; después GET /api/usuarios/19 respondió 404. Si las respuestas estuvieran hardcodeadas, el id no cambiaría, el taller no aparecería en el listado y el DELETE no afectaría las consultas siguientes.
- La información vive en MySQL (base mierda_batiz) y persiste entre peticiones; el backend Express solo la consulta y la devuelve. La prueba con la boleta repetida (409) también confirma que el servidor consulta los datos reales.

# Conclusión

La práctica permitió comprobar el ciclo completo de una API REST: crear, listar, consultar, actualizar, validar y eliminar recursos, con códigos HTTP coherentes y evidencia en Postman. Las 24 peticiones respondieron el código esperado; las observaciones principales son de datos de prueba y la URL fija de eliminación.

El punto técnico a corregir es la validación del taller en asistencias para evitar el 500. Con el ajuste propuesto, la API respondería 404 y el cliente sabría que el taller no existe, sin exponer un error interno de MySQL.